

PIP Source Specification v0.1

A Popcorn2 IPC Protocol Source file (source file, with the `.pip` extension) consists of ASCII encoded text describing interfaces, types, and references to other source files.

```
<source-file> ::=  
    <import-statement>  
    | <interface-definition>  
    | <bitfield-definition>  
    | <enum-definition>
```

Import statement

An *import statement* begins with the keyword `import`, followed by a file path enclosed in either angle brackets (`<>`) or double quotes (`" "`).

```
<import-statement> ::= "import" ("<" <path> ">") | ("\" <path> "\" )
```

The file path may be either relative or absolute. In the case of a relative file path, when enclosed in double quotes, the path shall be looked up relative to the current file, and when enclosed in angle brackets shall be looked up in a list of directories provided by the user. The mechanism for providing the directories is implementation defined by the compiler.

Double quotes and angle brackets may appear in a path, in which case they must be escaped by prefixing them with a backslash (`\` or `\>`). Backslashes may appear in a path in which case they must also be escaped by prefixing them with a backslash (`\\`).

Identifiers

Identifiers consist of letters, numbers, and underscores (`_`). *Identifiers* must begin with a letter.

```
<identifier> ::= <letter> (<letter> | <digit> | "_")*
```

Identifiers must not be a builtin *keyword*, as defined in the below list.

- `i8`, `i16`, `i32`, `i64`, `isize`
- `u8`, `u16`, `u32`, `u64`, `usize`
- `n8`, `n16`, `n32`, `n64`, `nsize`
- `import`, `interface`, `enum`, `bitfield`
- `func`, `reserve`
- `str`, `mem`, `impl`
- `err`

UIDs

UIDs consist of numbers, letters in the range A-F, and dashes (`-`). These must be in the format `XXXXXXXX-XXXX-XXXX-XXXX` where each `X` is either a number or letter in the range A-F.

Types

Types consist of *builtin types*, listed below, and *user defined types*. *User defined types* are identified by an identifier, which refers to an *in scope* type. A *type* is *in scope* if either:

- It is defined, before the usage, by a *bitfield definition* or an *enum definition* in the current source file.
- It is defined by a *bitfield definition* or an *enum definition* in a separate source file, which is *imported* into the current source file before the usage. This does apply recursively.

A source file is ill-formed if a *type* is referenced when not *in scope*.

A *builtin type* is a type defined in the below table.

Keyword	Description	Register count
<code>i8</code> , <code>i16</code> , <code>i32</code> , <code>i64</code>	Signed 8-, 16-, 32-, and 64-bit integers	1
<code>u8</code> , <code>u16</code> , <code>u32</code> , <code>u64</code>	Unsigned 8-, 16-, 32-, and 64-bit integers	1
<code>n8</code> , <code>n16</code> , <code>n32</code> , <code>n64</code>	Natural 8-, 16-, 32-, and 64-bit integers, consisting of the subset of numbers representable in a signed integer that are greater than or equal to zero	1
<code>isize</code> , <code>usize</code> , <code>nsize</code>	Signed, unsigned and natural architecture width integer	1
<code>str</code> , <code>outstr</code>	UTF-8 encoded text	2
<code>mem</code> , <code>outmem</code>	Arbitrary array of bytes	2
<code>err</code>	An error code	1
<code>impl</code>	An object type (see below)	1

Object type

An object type consists of the keyword `impl` followed by a list of *interfaces* in angle bracket (`<>`) separated by pluses (`+`). Each *interface* may be referenced by an *interface identifier*, or a *UID* in double quotes (`" "`). An *interface* referenced by an *interface identifier* must be *in scope*, using the same definition of *in scope* as for *types*.

```
<object-type> ::=
  "impl" "<"
  (<interface-identifier> | ("\" <uid> "\"))
  ("+" (<interface-identifier> | ("\" <uid> "\"))) *
  ">"
```

Interface definitions

An *interface definition* begins with the keyword `interface` followed by the interface name in double quotes. This is followed followed by an interface identifier, consisting of *identifiers* separated by dots (`.`). The *interface identifier* must not end with a dot. The *interface identifier* may be suffixed by an 'at' (`@`) followed by a numeric version number. The *interface identifier* is followed by an *interface block* enclosed in curly braces (`{ }`).

```
<interface-definition> ::= "interface" <interface-identifier> "{" <interface-block> "}"

<interface-identifier> ::= <identifier> ( "." <identifier> ) * [ "@" <integer> ]
```

The *interface block* is an array of functions and reserved items, separated by semicolons (`;`). Each item begins with either the keyword `func` or `reserve` followed by an 'at' (`@`) and a numeric identifier. The numeric identifier must be equal to or greater than zero, and less than 2^{32} . A source file is ill-formed if the numeric identifiers do not begin at zero, or if there are unused numbers less than the largest used identifier (see appendix for examples).

```
<interface-block> ::= ((<function> | <reserve>) ";" ) *

<reserve> ::= "reserve" "@" <integer> <identifier>

<function> ::= "func" "@" <integer> <identifier> <function-type>
```

In a function item, the numeric identifier is followed by an *identifier* and a *function type*, consisting of a list of comma separated *parameters* enclosed in parentheses (`()`). This may be followed by a *return type*, prefixed by an arrow (`->`).

A *parameter* consists of an *identifier*, a colon (`:`), and a *type*. The optional *return type* consists of an optional *success type*, an exclamation point (`!`), and an optional failure type. A source file is ill formed if there is neither a *success type* nor a *failure type*.

Builtin types are valid as return types in the following cases.

Builtin type	Success type alone	Success type with failure type	Failure type
i8, i16, i32, i64, isize	Valid		
u8, u16, u32, u64, usize	Valid		
n8, n16, n32, n64, nsize	Valid	Valid	
str, outstr			
mem, outmem			
err			Valid
impl	Valid	Valid	

All types are valid in parameters.

```
<function-type> ::= "(" (<identifier> ":" <type> ",") * ")" [ "->" <success-type> "!" <failure-type> ]
```

Register count

For each function item, a *total register count* is calculated. This is calculated by summing the *register count* for each parameter. A source file is ill formed if the *total register count* exceeds 3.

Type definitions

Enumerations

An *enum* definition begins with the keyword `enum`, followed by an *identifier*, an *underlying type* enclosed in parentheses (`()`), and an *enum block* enclosed in curly braces (`{ }`). The integer type must have a fixed width - a type of `isize`, `usize`, `nsize`, or any non integer type makes the source file ill formed.

The *enum block* consists of *enumerations* separated by commas, where each *enumeration* consists of an *identifier* and a number, separated by an 'at' (`@`). If the number is outside of the range of the *underlying type* of the *enum*, the source file is ill formed. If a number is repeated, the source file is ill formed.

Enums have the same *register count* as the *underlying type*. *Enums* may be used as a return type where their *underlying type* is permitted as a return type.

```
<enum-definition> ::= "enum" <identifier> "(" <integer-type> ")" "{" <enum-block> "}"

<enum-block> ::= (<enumeration> ",") *

<enumeration> ::= <identifier> "@" <number>
```

Bitfield

A *bitfield* definition begins with the keyword `bitfield`, followed by an *identifier*, an *underlying type* enclosed in parentheses (`()`), and a *bitfield block* enclosed in curly braces (`{}`). The integer type must have a fixed width, and be unsigned - a type of `isize`, `usize`, `nsize`, any signed type, or any non integer type makes the source file ill formed.

The *bitfield block* consists of *bits* separated by commas, where each *bit* consists of an *identifier* and a number, separated by an 'at' (`@`). If the number is larger than the number of bits in *underlying type* of the *bitfield*, the source file is ill formed. If a number is repeated, the source file is ill formed.

Bitfields have the same *register count* as the *underlying type*. *Bitfields* may be used as a return type where their *underlying type* is permitted as a return type.

```
<bitfield-definition> ::= "bitfield" <identifier> "(" <integer-type> ")" "{" <bitfield-block> "}"

<bitfield-block> ::= (<bit> ",")*

<bit> ::= <identifier> "@" <number>
```

Appendix A: Example source files

Valid source file

```
interface core.io.write@0 {
    func@0 write(data: mem) -> nsize!err;
    reserve @1;
    func@2 put(byte: u8) -> !err;
}

interface core.io.read@0 {
    func@0 read(len: usize, data: outmem) -> !err;
    func@1 get() -> u8!err;
}

interface core.fs.directory {
    func@0 open(index: usize) -> impl<core.io.read@0 + core.io.write@0>!err;
}

interface core.disk {
    func@0 get_partition(index: usize) -> impl<"00000000-00000000-0000-0003">!err;
}

enum seek_ty(n8) {
    start@0,
    current@1,
    end@2
}

interface core.io.seek@0 {
    func@0 seek(ty: seek_ty, amount: isize) -> !err;
}

bitfield keyboard_lights(u8) {
    caps_lock@0,
    num_lock@1,
    scroll_lock@2
}
```

Ill formed source files

```
interface core.io.write@0 {
    func@0 write(data: mem) -> nsize!err;
    func@2 put(byte: u8) -> !err;
}
```

There is no function defined with a numeric identifier of 1 .

```
interface core.io.seek@0 {
    func@0 seek(ty: seek_ty, amount: isize) -> !err;
}

enum seek_ty(n8) {
    start@0,
    current@1,
    end@2
}
```

The type `seek_ty` is referenced before definition.

```
interface bad {  
    func@0 a(data: mem, name: str);  
    func@1 b(data: mem, val1: u8, val2: u8);  
}
```

All functions have a *total register count* greater than 3.

```
interface no {  
    func@0 help() -> mem;  
}
```

`mem` used as a return type.